Chapter 7 — Structured Output & Tool Use

7.1 JSON Mode and Schema-Constrained Output

Conceptual Deep Dive

Free text is hard for programs to consume. Levels of control, weakest to strongest: (1) prompt asks for JSON; (2) **JSON mode** guarantees syntactically valid JSON but not your schema; (3) **structured outputs / JSON Schema mode** guarantees schema conformance (enforced by constrained decoding on the provider side); (4) self-hosted grammar-constrained decoding (7.3). Even with guaranteed shape, *values* can be wrong, so keep semantic validation. Schema design tips: flat, small enums, descriptive field names and `description`s (they act as prompts), optional → explicit `null`, put reasoning fields *before* answer fields so the model thinks first.

Cross-Connection Scenarios

Pydantic (3.3) generates schemas; evals check field-level accuracy (10); extraction pipelines feed RAG metadata (9).

Production Anti-Patterns

- Parsing with regex or `json.loads` on raw text without fallback.
- Giant, deeply nested schemas that reduce accuracy.
- Answer field before reasoning field.

Minimal Working Code

```
from typing import Literal
from pydantic import BaseModel, Field

class Invoice(BaseModel):
    reasoning: str = Field(description="Brief justification, written first")
    vendor: str
    total: float = Field(ge=0)
    currency: Literal["USD", "EUR", "GBP", "OTHER"]

schema = Invoice.model_json_schema()
print(list(schema["properties"]))  # pass `schema` as response_format JSON Schema
```

7.2 Validation, Repair and Retry Loops

Conceptual Deep Dive

Pattern: generate → validate with Pydantic → on failure feed the *validation error* back to the model ("fix these errors") → retry up to N times → fall back (human review / default). Libraries: **Instructor**, **PydanticAI**, LangChain output parsers. Track repair rate as a quality metric; a rising rate signals model drift or prompt regression. Use lenient parsing for markdown fences (```json) and trailing commas via repair helpers, but never silently coerce semantics.

Cross-Connection Scenarios

Retries/backoff (4.3), tracing of repairs (14), CI fixtures of malformed outputs (15).

Production Anti-Patterns

- Infinite repair loops.
- Retrying with identical prompt/temperature (same failure).
- Swallowing validation errors, passing partial objects downstream.

Minimal Working Code

```
import json
from pydantic import BaseModel, ValidationError

FENCE = "`" * 3  # models often wrap JSON in a markdown fence

def structured_call(llm, prompt: str, model: type[BaseModel], max_retries: int = 3):
    messages = [prompt]
    last_err = ""
    for _ in range(max_retries):
        raw = llm("\n".join(messages))
        raw = raw.strip().removeprefix(FENCE + "json").removesuffix(FENCE).strip()
        try:
            return model.model_validate(json.loads(raw))
        except (json.JSONDecodeError, ValidationError) as e:
            last_err = str(e)
            messages.append(f"Your previous output was invalid:\n{last_err}\nReturn corrected JSON o
nly.")
    raise ValueError(f"could not obtain valid output: {last_err}")
```

7.3 Grammar-Constrained Decoding: Outlines, Guidance, XGrammar

Conceptual Deep Dive

At each decoding step the model produces logits over the vocabulary. Constrained decoding computes the set of tokens allowed by a grammar/regex/JSON Schema given the text so far, sets all other logits to $-\infty$, then samples — so invalid output is *impossible*. Implementations: **Outlines** (compiles regex/JSON Schema to a finite-state machine indexed over the tokenizer vocabulary), **Guidance** (interleaves templates and generation with a grammar), **XGrammar** (fast pushdown-automaton grammar engine, integrated in vLLM/SGLang/TensorRT-LLM; handles context-free grammars and caches token masks), **llama.cpp GBNF** grammars. Caveats: tokenizer/grammar boundary mismatches, compile-time cost for complex schemas, and possible quality drop if the grammar forces unnatural continuations — keep a reasoning field.

Cross-Connection Scenarios

Serving engines (13) expose guided_json/structured-output options; tool calling on open models (12) relies on it.

Production Anti-Patterns

- Compiling a new grammar per request (cache it).
- Over-constraining so the model cannot express uncertainty (add null/"unknown").
- Assuming valid JSON means correct content.

Minimal Working Code

```
import math

def mask_logits(logits: list[float], allowed_ids: set[int]) -> list[float]:
    return [z if i in allowed_ids else -math.inf for i, z in enumerate(logits)]

# Toy FSM that only permits the strings "yes" / "no" (token ids are illustrative)
VOCAB = {0: "yes", 1: "no", 2: "maybe", 3: "<eos>"}
def allowed_after(prefix: list[int]) -> set[int]:
    return {3} if prefix else {0, 1}
```

```
# Real usage (outlines):  generator = outlines.generate.json(model, PydanticModel)
# vLLM: SamplingParams(guided_decoding=GuidedDecodingParams(json=schema))
print(mask_logits([0.2, 0.1, 5.0, 0.0], allowed_after([])))
```

7.4 Function Calling and Tool Definitions

Conceptual Deep Dive

You describe tools with a name, description and JSON Schema parameters. The model replies with a **tool call** (name + arguments) instead of text; *your code* executes it and returns the result as a tool message; the model then continues. Models can request **parallel** tool calls. The description is the single biggest lever on correct tool selection: say *when to use it and when not to*. Standards: provider function-calling APIs and the **Model Context Protocol (MCP)**, an open protocol where servers expose tools/resources/prompts to any compliant client. Treat arguments as untrusted input: validate, authorise per user, rate-limit, and sandbox.

Cross-Connection Scenarios

ReAct (6.3) is the text-based ancestor; agent loops (11); injection and permission design (6.5, 14).

Production Anti-Patterns

- Exposing a raw run_sql or shell tool with admin credentials.
- Dozens of overlapping tools (selection accuracy collapses; retrieve tools dynamically).
- Returning huge tool outputs that flood context.

Minimal Working Code

```
import json
from pydantic import BaseModel

class WeatherArgs(BaseModel):
    city: str

def get_weather(args: WeatherArgs) -> dict:
    return {"city": args.city, "temp_c": 21}

REGISTRY = {"get_weather": (WeatherArgs, get_weather)}

TOOL_SPECS = [{
    "name": "get_weather",
    "description": "Current weather for a city. Use only for weather questions.",
    "parameters": WeatherArgs.model_json_schema(),
}]

def execute(tool_call: dict) -> str:
    model, fn = REGISTRY[tool_call["name"]]
    try:
        args = model.model_validate(json.loads(tool_call["arguments"]))
        return json.dumps(fn(args))
    except Exception as e:
        return json.dumps({"error": str(e)})   # let the model self-correct

print(execute({"name": "get_weather", "arguments": '{"city": &quo
t;Paris"}'}))
```

7.5 Multi-Tool Orchestration

Conceptual Deep Dive

Orchestration concerns: **routing** (which tool), **sequencing** (dependencies between calls), **parallelism** (independent calls run concurrently with asyncio.gather), **error handling** (return structured errors the model can act on), **budgeting** (max calls/tokens/time), and **result shaping** (truncate, summarise, paginate). Workflows (fixed graph with LLM steps) are more reliable than fully autonomous loops; choose agents only when the path cannot be predetermined. For many tools, use tool retrieval (embed tool descriptions) or hierarchical routers.

Cross-Connection Scenarios

LangGraph/agents (11), idempotency for side effects (4.5), eval of tool-selection accuracy (10).

Production Anti-Patterns

- Serial execution of independent calls (latency).
- No per-tool timeouts.
- Not recording tool-call traces.

Minimal Working Code

```
import asyncio

async def run_tool(name, args, registry, timeout=10):
    try:
        return await asyncio.wait_for(registry[name](**args), timeout)
    except asyncio.TimeoutError:
        return {"error": f"{name} timed out"}
    except Exception as e:
        return {"error": str(e)}

async def run_parallel(calls, registry, max_calls=8):
    calls = calls[:max_calls]
    return await asyncio.gather(*(run_tool(c["name"], c["args"], registry) for c in c
alls))

async def a(x): return {"a": x}
async def b(y): return {"b": y}
print(asyncio.run(run_parallel([{"name": "a", "args": {"x": 1}},
{"name": "b", "args": {"y": 2}}],
                               {"a": a, "b": b})))
```

7.6 Schema Design and Evolution

Conceptual Deep Dive

Schemas are APIs between the model and your code, so version them. Design rules: use enums for closed sets, Literal for discriminators, Optional with explicit null for absent data, bounded numbers and string lengths, and short description strings that act as inline instructions. For variants use **discriminated unions** (type field) rather than loosely typed objects. Avoid recursion depth and huge enums where the provider's constrained decoding has limits. Evolve by adding optional fields first, deprecating before removing, and storing schema_version with every persisted result. Edge cases: models inventing enum values when unconstrained, numeric strings versus numbers ("1,200"), dates in ambiguous formats (use ISO 8601 and validate), and unicode normalisation in identifiers.

Cross-Connection Scenarios

Pydantic (3.3), validation loops (7.2), regression tests on schema changes (15.2).

Production Anti-Patterns

- Changing a field's meaning without bumping the version.
- Free-text fields where an enum would do.
- Parsing dates and currency with lenient regex.

Minimal Working Code

```
from typing import Annotated, Literal, Union
from pydantic import BaseModel, Field

class Refund(BaseModel):
    type: Literal["refund"]
    order_id: str
    amount: float = Field(gt=0, le=10_000)

class Escalate(BaseModel):
    type: Literal["escalate"]
    reason: str = Field(max_length=300)

Action = Annotated[Union[Refund, Escalate], Field(discriminator="type")]

class Decision(BaseModel):
    schema_version: Literal[1] = 1
    action: Action

print(Decision.model_validate({"action": {"type": "refund", "order_id&
quot;: "o1", "amount": 12.5}}))
```

```
from datetime import date
from pydantic import BaseModel, field_validator

class Event(BaseModel):
    day: date
    cost: float

    @field_validator("cost", mode="before")
    @classmethod
    def parse_money(cls, v):
        if isinstance(v, str):
            return float(v.replace("$", "").replace(",", ""))
        return v

print(Event(day="2026-10-22", cost="$1,200.50"))
```

7.7 Tool Safety, Permissions and Human Approval

Conceptual Deep Dive

Classify tools by blast radius: **read-only**, **reversible write**, **irreversible/external** (send email, payment, deploy). Policy: read-only tools run freely; reversible writes run with logging and undo; irreversible actions require explicit human approval, shown with the exact arguments (not the model's paraphrase). Enforce authorisation in code with the end user's identity, validate arguments against allow-lists (hosts, tables, paths), cap amounts and rates, and run execution tools in sandboxes. Return errors that are informative to the model but never leak secrets. Edge cases: path traversal in file tools (../../), SSRF via URL-fetch tools (internal IPs, cloud metadata endpoints), SQL tools that accept arbitrary statements, and argument smuggling through nested JSON.

Cross-Connection Scenarios

Injection defence (6.5, 14.2), agent approvals (11.1), idempotency keys (4.5).

Production Anti-Patterns

- Approval prompts that show only a natural-language summary.
- A URL-fetch tool with no private-network blocking.
- File tools without a root-directory jail.

Minimal Working Code

```
import ipaddress
import socket
from pathlib import Path
from urllib.parse import urlparse

ROOT = Path("/srv/workspace").resolve()

def safe_path(user_path: str) -> Path:
    p = (ROOT / user_path).resolve()
    if ROOT not in p.parents and p != ROOT:
        raise PermissionError("path escapes workspace")
    return p

def safe_url(url: str) -> str:
    u = urlparse(url)
    if u.scheme not in ("http", "https") or not u.hostname:
        raise ValueError("bad url")
    for info in socket.getaddrinfo(u.hostname, None):
        ip = ipaddress.ip_address(info[4][0])
        if ip.is_private or ip.is_loopback or ip.is_link_local or ip.is_reserved:
            raise PermissionError("private address blocked")
    return url
```

```
RISK = {"search": "read", "draft_email":
 "reversible", "send_email": "irreversible"}

def execute_with_policy(name, args, fn, approve):
    risk = RISK.get(name, "irreversible")        # unknown tools default to the strictest tier
    if risk == "irreversible" and not approve(name, args):
        return {"status": "denied_by_user"}
    return {"status": "ok", "result": fn(**args)}

print(execute_with_policy("send_email", {"to": "a@b.c"}, lambda **k: "
sent", lambda n, a: False))
```

7.8 Model Context Protocol (MCP): Servers, Clients and Security

Conceptual Deep Dive

MCP is an open protocol that standardises how applications give models access to tools, resources and prompts. An **MCP server** exposes: *tools* (callable functions with JSON Schema), *resources* (readable data such as files or records), and *prompts* (reusable templates). An **MCP client** inside a host application (IDE, chat app, agent) discovers a server's capabilities, lists tools, and forwards model-requested calls. Transports: **stdio** (local subprocess) and **streamable HTTP** (remote, with authentication, typically OAuth). Benefits: write an integration once and reuse it across hosts; clear separation between model, host and tool. Risks: servers are third-party code with real privileges. Treat tool descriptions as untrusted prompt content (**tool poisoning**), pin and review servers, scope credentials narrowly, require user confirmation for sensitive tools, and avoid mixing an untrusted server with a server holding private data in one agent session. Edge cases: servers that change tool definitions after approval ("rug pull"), name collisions across servers, very large tool lists degrading tool selection, and long-running tool calls needing progress notifications.

Cross-Connection Scenarios

Function calling (7.4), tool safety (7.7), agent protocols (11.10), injection defence (6.5, 14.2).

Production Anti-Patterns

- Installing unreviewed community servers with broad filesystem or credential access.
- Exposing dozens of tools at once instead of curating per task.
- Passing user OAuth tokens through to every server.

Minimal Working Code

```
# pip install mcp
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("orders")

@mcp.tool()
def get_order_status(order_id: str) -> dict:
    """Look up the status of an order by id. Read-only."""
    if not order_id.startswith("ord_"):
        raise ValueError("order_id must start with 'ord_'")
    return {"order_id": order_id, "status": "shipped"}

@mcp.resource("policy://refunds")
def refund_policy() -> str:
    """Current refund policy text."""
    return "Refunds are available within 30 days of delivery."

if __name__ == "__main__":
    mcp.run()          # stdio transport by default
```

```
import hashlib
import json

def tool_fingerprint(tool_defs: list[dict]) -> str:
    canonical = json.dumps(sorted(tool_defs, key=lambda t: t["name"]), sort_keys=True)
    return hashlib.sha256(canonical.encode()).hexdigest()

class ToolPin:
    """Detect changed tool definitions after a server was approved."""
    def __init__(self):
        self.approved: dict[str, str] = {}

    def approve(self, server: str, tool_defs: list[dict]):
        self.approved[server] = tool_fingerprint(tool_defs)

    def check(self, server: str, tool_defs: list[dict]) -> bool:
        return self.approved.get(server) == tool_fingerprint(tool_defs)

pin = ToolPin()
pin.approve("orders", [{"name": "get_order_status", "description"
: "v1"}])
print(pin.check("orders", [{"name": "get_order_status", "description&q
uot;: "v1 + exfiltrate secrets"}]))
```

7.9 Streaming Structured Output and Partial JSON

Conceptual Deep Dive

Waiting for a complete JSON object hurts perceived latency. **Partial JSON parsing** extracts usable fields as tokens arrive: close open strings/brackets heuristically, or use an incremental parser. UI pattern: render fields progressively (title first, details later) while validating the final object strictly. For tool calls, argument JSON also arrives in fragments across chunks, so accumulate per call index and only execute once the call is complete and valid. Order schema fields so the most useful or cheapest-to-display ones come first, but keep reasoning fields before final answers for quality. Edge cases: escaped characters split across chunks, numbers that look complete but continue ("12" then "5"), duplicate keys, and the model emitting text before the JSON.

Cross-Connection Scenarios

Streaming transports (4.2), validation loops (7.2), tool-call parsing (7.4), UI patterns (2.8).

Production Anti-Patterns

- Executing a tool after seeing the first closing brace of a nested object.

- Displaying unvalidated partial values as final facts.
- Crashing the stream on one malformed fragment instead of buffering.

Minimal Working Code

```
import json

def close_partial_json(s: str) -> str:
    """Best-effort completion of a truncated JSON string (for display only)."&q
uot;
    stack, in_str, esc = [], False, False
    for ch in s:
        if in_str:
            if esc:
                esc = False
            elif ch == "\\":
                esc = True
            elif ch == "":
                in_str = False
        else:
            if ch == "":
                in_str = True
            elif ch in "{[":
                stack.append("}" if ch == "{" else "]")
            elif ch in "}]" and stack:
                stack.pop()
    tail = "" if in_str else ""
    return s + tail + "".join(reversed(stack))

def best_effort(s: str):
    s2 = close_partial_json(s).rstrip().rstrip(",")
    try:
        return json.loads(s2)
    except json.JSONDecodeError:
        return None

buf = ""
for chunk in ['{"title": "Q3 ', 'report", "items": [{"n": 1}, {&q
uot;n"', ': 2}], "done": true}']:
    buf += chunk
    print(best_effort(buf))
```

```
class ToolCallAssembler:
    """Collect streamed tool-call fragments keyed by call index."""
    def __init__(self):
        self.calls: dict[int, dict] = {}

    def feed(self, delta: dict):
        c = self.calls.setdefault(delta["index"], {"name": "", "args&q
uot;: ""})
        c["name"] += delta.get("name", "")
        c["args"] += delta.get("arguments", "")

    def ready(self):
        out = []
        for i, c in self.calls.items():
            try:
                out.append((c["name"], json.loads(c["args"])))
            except json.JSONDecodeError:
                continue
        return out

a = ToolCallAssembler()
for d in [{"index": 0, "name": "get_weather", "arguments": '{&quo
t;city": "Pa'}, {"index": 0, "arguments": 'ris"}']:
```

```
    a.feed(d)
print(a.ready())
```